

LECTURE 6: SPECIFICATIONS

Dr. Ehab Essa
Computer Science Department,
Mansoura University

WHY SPECIFICATIONS?

Specifications are a detailed description of what a software component (like a function or module) is supposed to do.

It serves as a reference or blueprint for the expected behavior of the software. It may detail inputs, outputs, and possible side effects, among other things..

It's impossible to delegate responsibility for implementing a method without a specification.

Many of the **nastiest bugs** in programs arise because of **misunderstandings** about behavior at the interface between the caller of a code and the implementation.

When the program fails, it's hard to determine where the error is. **Precise specifications** in the code let you apportion blame to code fragments, and can spare you the puzzling over where a fix should go.

WHY SPECIFICATIONS?

The **specification acts as a contract**: the implementer is responsible for meeting the contract, and a client that uses the method can rely on the contract.

Specifications **are good for the client** of a method because they spare the task of reading code.

Specifications **are good for the implementer** of a method because they give the implementer freedom to change the implementation without telling clients.

Sometimes by having **a weak specification** that leaves vague some of the things that the function might do in some circumstances, you can choose **a much more efficient implementation**.

Particular **precondition** may rule out certain states, certain ways you might call a method that would have forced the implementer to make some kind of expensive check that's no longer necessary.

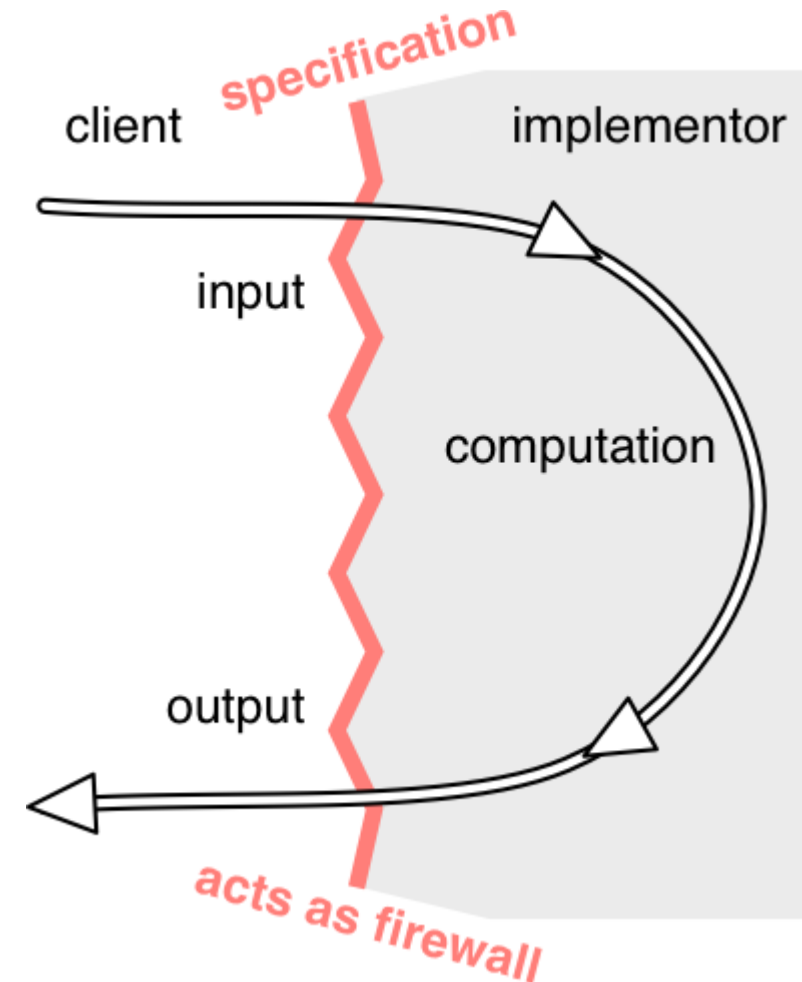
WHY SPECIFICATIONS?

The **contract** acts as a firewall between client and implementer.

It shields the client from the details of the workings of the unit — you don't need to read the source code of the procedure if you have its specification.

And it shields the implementer from the details of the usage of the unit. The implementer doesn't have to ask every client how they plan to use the unit.

This firewall results in decoupling, allowing the code of the unit and the code of a client to be changed independently, so long as the changes respect the specification — each obeying its obligation.



BEHAVIORAL EQUIVALENCE

Behavioral equivalence, is whether we could substitute one implementation for the other.

Consider these two methods. Are they the same or different?

```
static int findFirst(int[] arr, int val) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr[i] == val) return i;  
    }  
    return arr.length;  
}
```

```
static int findLast(int[] arr, int val) {  
    for (int i = arr.length - 1; i >= 0; i--) {  
        if (arr[i] == val) return i;  
    }  
    return -1;  
}
```

The code is different, Not only do these methods have different code, they actually have different behavior:

- when val is missing, findFirst returns the length of arr and findLast returns -1;
- when val appears twice, findFirst returns the lower index and findLast returns the higher.

But when val occurs at exactly one index of the array, the two methods behave the same: they both return that index.

BEHAVIORAL EQUIVALENCE

The notion of equivalence is in the eye of the client.

For example, whenever **clients** call the method, **they will be passing in an arr with exactly one element val**. For such clients, these two methods are the **same**, and we could switch from one implementation to the other without issue.

In order to substitute one implementation for another, and to know when this is acceptable, we need **a specification** that states exactly what the client depends on.

In this case, our specification might be:

```
static int find(int[] arr, int val)
```

requires: **val occurs exactly once in arr**

effects: returns index i such that $\text{arr}[i] = \text{val}$

SPECIFICATION STRUCTURE

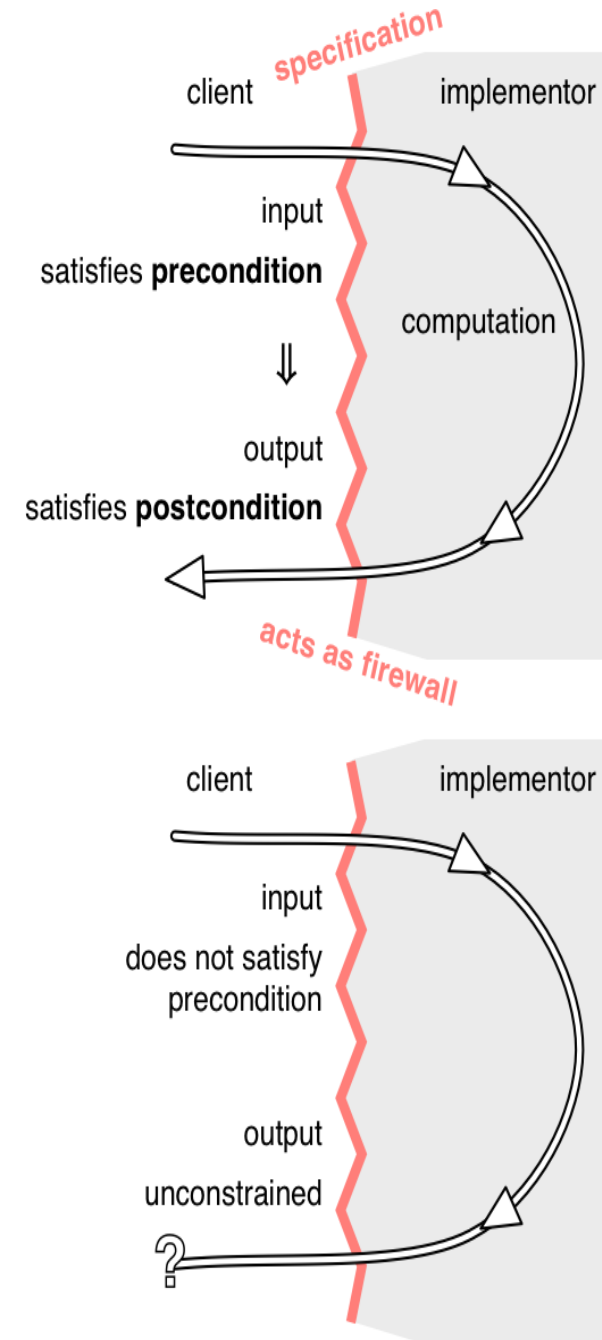
A specification of a method consists of two clauses:

- a precondition, indicated by the keyword **requires**
- a postcondition, indicated by the keyword **effects**

The **precondition** is an obligation on the client (i.e., the caller of the method). It's a condition over the state in which the method is invoked.

The **postcondition** is an obligation on the implementer of the method. If the precondition holds for the invoking state, the method is obliged to obey the postcondition, by returning appropriate values, throwing specified exceptions, modifying or not modifying objects, and so on.

If the **precondition does not hold** when the method is called, the implementation is **not bound by the postcondition**. It is free to do anything, including not terminating, throwing an exception, returning arbitrary results, making arbitrary modifications, etc.



SPECIFICATIONS IN JAVA

Some languages (notably **Eiffel**) incorporate preconditions and postconditions as a fundamental part of the language, as expressions that the runtime system (or even the compiler) can automatically check to enforce the contracts between clients and implementers. This is called "**Design by Contract**".

The main components of **DbC** are: Preconditions, Postconditions and Invariants.

- Invariants describe conditions that remain consistent before and after any method is invoked on that object. For example, if you have a class representing a circle, an invariant might be that its radius is always non-negative.

Java does not automatically enforce DbC, it depends on human beings to check it and guarantee it.

In Java, the contract must be described in a comment preceding the method,

Java has a convention for documentation comments, in which

- parameters are described by `@param` clauses
- and results are described by `@return` and `@throws` clauses.

You should put the preconditions into `@param` where possible, and postconditions into `@return` and `@throws`.

SPECIFICATIONS IN JAVA

So a specification like this:

```
static int find(int[] arr, int val)
    requires: val occurs exactly once in arr
    effects:  returns index i such that arr[i] = val
```

... might be rendered in Java like this:

```
/**
 * Find a value in an array.
 * @param arr array to search, requires that val occurs exactly once
 *           in arr
 * @param val value to search for
 * @return index i such that arr[i] = val
 */
static int find(int[] arr, int val)
```

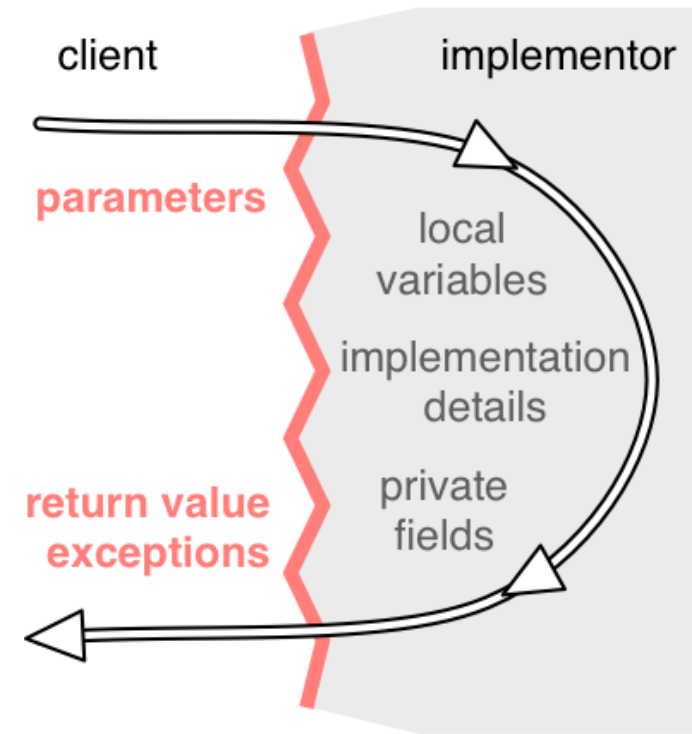
WHAT A SPECIFICATION MAY TALK ABOUT

A specification of a method can talk about:

the parameters and return value of the method,

but it should never talk about local variables of the method or private fields of the method's class.

You should consider the implementation invisible to the reader of the spec.



TESTING AND SPECIFICATIONS

it's important to note that even glass box tests must follow the specification.

Your test cases should not count on that behaviour (implementation). Test cases must obey the contract, just like every other client.

```
static int find(int[] arr, int val)
  requires: val occurs in arr
  effects:  returns index i such that arr[i] = val
```

```
int[] array = new int[] { 7, 7, 7 };
assertEquals(0, find(array, 7)); // bad test case: violates the spec
assertEquals(7, array[find(array, 7)]); // correct
```

This spec has a **strong precondition** in the sense that val is required to be found; and

it has a fairly **weak postcondition** in the sense that if val appears more than once in the array, this **specification says nothing about which particular index of val is returned**.

Even if you implemented find so that it always returns the lowest index, your test case can't assume that specific behaviour.

TESTING AND SPECIFICATIONS

Similarly, even if you **implemented find so that it (sensibly) throws an exception when val isn't found**, instead of returning some arbitrary misleading index, **your test case can't assume** that behavior, because it can't call find() in a way that violates the precondition.

So what does glass box testing mean, if it can't go beyond the spec?

- It means you are trying to find new test cases that exercise different parts of the implementation, but still checking those test cases in an implementation-independent way.

In unit testing, tests individual units or components of a software in isolation without external interactions.

A "unit" could be a function, method, procedure, or even a module, depending on the context. we write tests of each module of our program in isolation.

A good unit test is focused on just a single specification. Only test one written method.

A unit test for one method shouldn't fail if a different method fails to satisfy its spec.

TESTING AND SPECIFICATIONS

Good integration tests, tests that use a combination of modules, will make sure that our different methods have compatible specifications: callers and implementors of different methods are passing and returning values as the other expects.

Integration tests cannot replace systematically-designed unit tests.

```
/** @return the contents of the web page downloaded from url */
public static String getWebPage(URL url) { ... }

/** @return the words in string s, in the order they appear,
 *     where a word is a contiguous sequence of
 *     non-whitespace and non-punctuation characters */
public static List<String> extractWords(String s) { ... }

/** @return an index mapping a word to the set of URLs
 *     containing that word, for all webpages in the input set */
public static Map<String, Set<URL>> makeIndex(Set<URL> urls) {
    ...
    calls getWebPage and extractWords
    ...
}
```

SPECIFICATIONS FOR MUTATING METHODS

Specifications must describe side-effects —changes to mutable data — in the postcondition.

```
static boolean addAll(List<T> list1, List<T> list2)
    requires: list1 != list2
    effects:  modifies list1 by adding the elements of list2 to the end of
              it, and returns true if list1 changed as a result of call
```

First, look at the postcondition. It gives two constraints: the first telling us how list1 is modified, and the second telling us how the return value is determined.

Second, look at the precondition. It tells us that the behavior of the method if you attempt to add the elements of a list to itself is undefined.

SPECIFICATIONS FOR MUTATING METHODS

Here is another example of a mutating method:

```
static void sort(List<String> lst)
  requires: nothing
  effects: puts lst in sorted order, i.e.  $lst[i] \leq lst[j]$ 
           for all  $0 \leq i < j < lst.size()$ 
```

And an example of a method that does not mutate its argument:

```
static List<String> toLowerCase(List<String> lst)
  requires: nothing
  effects: returns a new list t where  $t[i] = lst[i].toLowerCase()$ 
```

Note: null is implicitly disallowed unless stated otherwise, we will also use the convention that mutation is disallowed unless stated otherwise.

In the **absence of a postcondition describing mutation, assume no mutation of the inputs.**

DESIGNING SPECIFICATIONS

INTRODUCTION

We'll look at different specs for similar behaviors, and talk about the tradeoffs between them:

How **deterministic** it is. Does the spec define only a single possible output for a given input, or allow the implementer to choose from a set of legal outputs?

How **declarative** it is. Does the spec just characterize what the output should be, or does it explicitly say how to compute the output?

How **strong** it is. Does the spec have a small set of legal implementations, or a large set?

Not all specifications we might choose for a module are equally useful, and we'll explore what makes some specifications better than others.

DETERMINISTIC VS. UNDERDETERMINED SPECS

Recall the two example implementations of find we began with in the previous reading:

```
static int findFirst(int[] arr, int val) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr[i] == val) return i;  
    }  
    return arr.length;  
}
```

The **first specification** is deterministic: when presented with a state satisfying the precondition, the outcome is completely determined. One valid input (val) has only one valid output (index i).

The **second specification** is underdetermined. It doesn't say which index is returned if val occurs more than once. It simply says that if you look up the entry at the index given by the returned value, you'll find val. This specification allows multiple valid outputs for the same input.

```
static int findLast(int[] arr, int val) {  
    for (int i = arr.length - 1; i >= 0; i--) {  
        if (arr[i] == val) return i;  
    }  
    return -1;  
}
```

```
static int findExactlyOne(int[] arr, int val)  
    requires: val occurs exactly once in arr  
    effects: returns index i such that arr[i] = val
```

```
static int findOneOrMore,AnyIndex(int[] arr, int val)  
    requires: val occurs in arr  
    effects: returns index i such that arr[i] = val
```

DECLARATIVE VS. OPERATIONAL SPECS

Operational specifications give a series of steps that the method performs; pseudocode descriptions are operational.

Declarative specifications don't give details of intermediate steps. Instead, they just give properties of the final outcome and how it's related to the initial state.

Almost always, **declarative specifications are preferable**.

They're usually shorter, easier to understand, and most importantly, they don't inadvertently expose implementation details that a client may rely on (and then find no longer hold when the implementation is changed).

One reason programmers sometimes lapse into operational specifications is because they're using the spec comment to explain the implementation for a maintainer. Don't do that. When it's necessary, use comments within the body of the method, not in the spec comment.

DECLARATIVE SPECS

For a given specification, there may be many ways to express it declaratively:

```
static boolean startsWith(String str, String prefix)
  effects: returns true if and only if there exists String suffix
           such that prefix + suffix = str
```

```
static boolean startsWith(String str, String prefix)
  effects: returns true if and only if there exists integer i
           such that str.substring(0, i) = prefix
```

```
static boolean startsWith(String str, String prefix)
  effects: returns true if the first prefix.length() characters of str
           are the characters of prefix, false otherwise
```

It's up to us to choose the clearest specification for clients and maintainers of the code.

STRONGER VS. WEAKER SPECS

Suppose you want to change a method — either how its implementation behaves, or the specification itself. There are already clients that depend on the method's current specification. **How do you compare the behaviors of two specifications to decide whether it's safe to replace the old spec with the new spec?**

A specification $S2$ is **stronger than** or equal to a specification $S1$ if

- $S2$'s precondition is **weaker than** or equal to $S1$'s, and
- $S2$'s postcondition is **stronger than** or equal to $S1$'s for the states that satisfy $S1$'s precondition.

If this is the case, **it's safe to replace $S1$ with $S2$ in your program.**

These two rules embody several ideas. They tell you that you can **always weaken the precondition; placing fewer demands on a client will never upset them.** And you can always strengthen the post-condition, which means making **more promises.**

STRONGER VS. WEAKER SPECS

For example, this spec for `find`:

```
static int findExactlyOne(int[] a, int val)
  requires: val occurs exactly once in a
  effects:  returns index i such that a[i] = val
```

can be replaced with:

```
static int findOneOrMore,AnyIndex(int[] a, int val)
  requires: val occurs at least once in a
  effects:  returns index i such that a[i] = val
```

which has a weaker precondition. This in turn can be replaced with:

```
static int findOneOrMore,FirstIndex(int[] a, int val)
  requires: val occurs at least once in a
  effects:  returns lowest index i such that a[i] = val
```

which has a stronger postcondition.

What about this specification:

```
static int findCanBeMissing(int[] a, int val)
  requires: nothing
  effects:  returns index i such that a[i] = val,
           or -1 if no such i
```

DESIGNING GOOD SPECIFICATIONS

What makes a good method? Designing a method means primarily writing a specification.

About the form of the specification: it should obviously be succinct, clear, and well-structured, so that it's easy to read.

The specification should be coherent

- The spec shouldn't have lots of different cases. Long argument lists, deeply nested if-statements, and boolean flags are all signs of trouble. Consider this specification

```
static int sumFind(int[] a, int[] b, int val)  
    effects: returns the sum of all indices in arrays a and b at which  
              val appears
```

- Is this a well-designed procedure? Probably not: it's incoherent, since it does several things (finding in two arrays and summing the indexes) that are not really related. It would be better to use two separate procedures, one that finds the indexes, and the other that sums them.

DESIGNING GOOD SPECIFICATIONS

The specification should be strong enough

- **Clarity:** The specification should be clear and unambiguous, ensuring that anyone who reads it understands its intent without confusion. there's no room for multiple interpretations.
- **Succinctness:** While being comprehensive, the specification should be concise, avoiding unnecessary verbosity.
- **Completeness:** Cover all expected behaviors, including both the general and the edge or special cases.
- **Consistency:** Ensure that the specification doesn't contradict itself. Different parts should align and work together coherently.

DESIGNING GOOD SPECIFICATIONS

The specification should also be weak enough

- It should be realistic in its promises and acknowledge potential limitations.
- This helps managing expectations and ensures clients know the boundaries of what's being offered.
- The specification should say something much weaker: that it attempts to do the task, and what happens when fails or success.

```
static File open(String filename)  
    effects: opens a file named filename
```

- This is a bad specification. it's too strong, since there's no way it can guarantee to open a file and It lacks important details: is the file opened for reading or writing? Does it already exist or is it created?

The specification should use *abstract types* where possible

- Writing our specification with abstract types gives more freedom to both the client and the implementer. E.g. Using a List or Set rather than ArrayList or HashSet.

PRECONDITION OR POSTCONDITION?

Using Preconditions:

- If a method has a precondition, it traditionally expects the caller to ensure that this condition is satisfied before calling the method. If the caller fails to meet the precondition and still calls the method, unpredictable behavior or errors can result.

Specify preconditions as postconditions :

- many Java API classes prefer to specify their conditions as postconditions where they **throw unchecked exceptions** if the arguments are inappropriate. This approach is chosen to "fail fast" and help find bugs or incorrect assumptions in the calling code.

Should the method code check the precondition has been met before proceeding?

- If checking a precondition inside the method makes the method significantly slower or less efficient, then a precondition is often necessary but without checking inside in this case.
- For example binary search. If the method had to check every time whether the array is sorted, it would negate the performance benefits of binary search.

ABOUT ACCESS CONTROL

The decision to make a method **public** or **private** is actually a decision about the contract of the class.

Making a method public advertises it as a service that your class is willing to provide.

If you make all your methods public — including helper methods that are really meant only for local use within the class — then other parts of the program may come to depend on them, which will make it harder for you to change the internal implementation of the class in the future. Your code won't be as ready for change.

Making internal helper methods public will also add clutter to the visible interface your class offers. Keeping internal things private makes your class's public interface smaller and more coherent (meaning that it does one thing and does it well). Your code will be easier to understand .